

CHƯƠNG 2. LỚP VÀ ĐỐI TƯỢNG

TS. Phạm Minh Hoàn

Viện Công nghệ thông tin Kinh tế – Đại học Kinh tế Quốc dân

Email: hoanpm@neu.edu.vn

MỤC TIÊU

- Trình bày phương pháp định nghĩa lớp, thành phần lớp, phạm vi truy xuất thành phần, các hàm đặc biệt của lớp: hàm bạn, hàm tạo và hàm hủy.
- Khai báo đối tượng, mảng và con trỏ đối tượng.
- Cách thức truy nhập đến các thành phần của đối tượng, con trỏ mà mảng đối tượng.
- Trình bày khái niệm thành phần tĩnh, dữ liệu và phương thức tĩnh.

NỘI DUNG CHƯƠNG 2

- *Lớp*
 - *Định nghĩa*
 - *Thành phần dữ liệu*
 - *Thành phần phương thức*
 - *Từ khóa xác định phạm vi truy xuất*
 - *Con trỏ this*
 - *Hàm bạn*
 - *Hàm tạo, hàm hủy*
- *Đối tượng*
 - *Biến*
 - *Mảng và con trỏ*
 - *Toán tử tải bội (Operator overloading)*
- *Các thành phần tĩnh*
 - *Khái niệm*
 - *Dữ liệu tĩnh*
 - *Phương thức tĩnh*

LỚP

ĐỊNH NGHĨA

- Lớp là khái niệm trung tâm của lập trình hướng đối tượng, nó là sự mở rộng của các khái niệm cấu trúc (struct) của ngôn ngữ lập trình C.
- Ngoài các thành phần dữ liệu, lớp còn chứa các thành phần hàm, còn gọi là phương thức (method) hoặc hàm thành viên (member function).
- Lớp có thể xem như một kiểu dữ liệu các biến, mảng đối tượng. Từ một lớp đã định nghĩa, có thể tạo ra nhiều đối tượng khác nhau, mỗi đối tượng có vùng nhớ riêng.

LỚP

ĐỊNH NGHĨA

Cú pháp:

```
class tên_lớp
{
    private:
        [Khai_báo_các_thuộc_tính]
        [Định_nghĩa_các_hàm_thành_phần_(phương
            _thức)]
    public:
        [Khai_báo_các_thuộc_tính]
        [Định_nghĩa_các_hàm_thành_phần
            (phương_thức)]
};
```

LỚP

ĐỊNH NGHĨA

- *Thuộc tính là dữ liệu của lớp, phương thức là các hàm tác động lên dữ liệu của lớp đó được gọi là hàm của lớp.*
- *Dữ liệu và hàm thành viên được gọi chung là các thành phần của lớp.*

LỚP

THÀNH PHẦN DỮ LIỆU

- Khai báo các thuộc tính (dữ liệu) được thực hiện như khai báo biến có kiểu chuẩn hoặc kiểu ngoài chuẩn đã được định nghĩa trước (cấu trúc, hợp, lớp, ...).
- Thuộc tính của lớp không thể có kiểu chính của lớp đó, nhưng có thể là kiểu con trỏ của lớp này.

LỚP

THÀNH PHẦN PHƯƠNG THỨC (HÀM)

- Các hàm thành phần có thể được xây dựng bên trong hoặc bên ngoài định nghĩa lớp.
- Hàm thành phần đơn giản, có ít dòng lệnh sẽ được viết bên trong định nghĩa lớp như hàm thông thường.
- Hàm thành phần dài thì viết bên ngoài định nghĩa lớp.

LỚP

THÀNH PHẦN PHƯƠNG THỨC (HÀM)

- Cú pháp định nghĩa hàm thành phần ở bên ngoài lớp:

```
Kiểu_trả_về_của_hàm  Tên_lớp::Tên_hàm(khai báo các tham  
số)  
{  
    //Nội dung hàm  
}
```

Toán tử :: được gọi là **toán tử phân giải miền xác định**, được dùng để chỉ ra lớp mà hàm đó thuộc vào.

LỚP

THÀNH PHẦN PHƯƠNG THỨC (HÀM)

- *Hàm thành phần có thể không có giá trị trả về (kiểu void) hoặc có thể trả về một giá trị có kiểu bất kỳ, kể cả giá trị kiểu đối tượng, con trỏ đối tượng, tham chiếu đối tượng.*
- *Tham số của hàm thành phần có thể có kiểu bất kỳ: kiểu chuẩn, kiểu ngoài chuẩn, kiểu đối tượng của chính phương thức, con trỏ hoặc tham chiếu đến kiểu đối tượng này.*
- *Trong thân hàm thành phần, có thể sử dụng các thuộc tính của lớp, các hàm thành phần khác và các hàm tự do trong chương trình.*

LỚP

THÀNH PHẦN PHƯƠNG THỨC (HÀM)

○ Chú ý :

- Các thành phần dữ liệu khai báo là private nhằm bảo đảm nguyên lý che dấu thông tin, bảo vệ an toàn dữ liệu của lớp, không cho phép các hàm bên ngoài xâm nhập vào dữ liệu của lớp.
- Các hàm thành phần khai báo là public có thể được gọi tới từ các hàm thành phần public khác trong chương trình.

LỚP

THÀNH PHẦN PHƯƠNG THỨC (HÀM)

- Ví dụ: Định nghĩa lớp để mô tả và xử lý các điểm trên màn hình đồ họa. Lớp được đặt tên là DIEM.
 - Các thuộc tính của lớp gồm:
 - `int x;` // hoành độ (cột)
 - `int y;` // tung độ (hàng)
 - `int m;` // màu
 - Các phương thức:
 - Nhập dữ liệu của một điểm
 - Hiển thị một điểm
 - Ẩn một điểm

LỚP

THÀNH PHẦN PHƯƠNG THỨC (HÀM)

○ Ví dụ: Xây dựng lớp DIEM.

```
class DIEM
{
    private:
        int x,y,m;
    public:
        void nhapdl();
        void hien();
        void an()
        {
            putpixel(x,y,getbkcolor());
        }
};
```

LỚP

THÀNH PHẦN PHƯƠNG THỨC (HÀM)

- Ví dụ: Xây dựng lớp DIEM.

```
void DIEM::nhapdl()  
{  
    cout<<"\nNhap hoành do (cot) va  
    tung do (hang) cua diem:";  
    cin>>x>>y;  
    cout<<"\nNhap ma mau cua  
    diem:";  
    cin>>m;  
}
```

LỚP

THÀNH PHẦN PHƯƠNG THỨC (HÀM)

- Ví dụ: Xây dựng lớp DIEM.

```
void DIEM::hien()
{
    int mau_ht;
    mau_ht = getcolor();
    putpixel(x,y,m);
    setcolor(mau_ht);
}
```

LỚP

TỪ KHÓA XÁC ĐỊNH PHẠM VI TRUY XUẤT

- Các thành phần của lớp được tổ chức thành hai vùng: *vùng sở hữu riêng (private)* và *vùng dùng chung (public)* để quy định phạm vi sử dụng của các thành phần.
- Những thành phần thuộc vùng sở hữu riêng chỉ được sử dụng trong phạm vi của lớp, còn những thành phần thuộc vùng dùng chung có thể sử dụng cả ở trong và ngoài lớp.

LỚP

CON TRỎ THIS

- Mỗi hàm thành phần của lớp có một tham số ẩn, đó là con trỏ **this**. Con trỏ this trỏ đến từng đối tượng cụ thể.

```
void nhapsl()
```

```
{
```

```
    cout<<"\nNhap hoành do va tung do  
    cua diem:";
```

```
    cin >>this->x>>this->y ;
```

```
}
```

LỚP

CON TRỎ THIS

- Con trỏ **this** là tham số thứ nhất của hàm thành phần. Khi một lời gọi hàm thành phần được phát ra bởi một đối tượng thì tham số truyền cho con trỏ **this** chính là địa chỉ của đối tượng đó.
- Ví dụ: Xét một lời gọi tới hàm nhapsl():
 DIEM d1 ;
 d1.nhapsl();
Trong trường hợp này của d1 thì this =&d1.
Do đó:
 this -> nhapsl() chính là d1.nhapsl()

LỚP

HÀM BẠN

- Hàm bạn không phải là hàm thành phần của lớp.
- Việc truy nhập tới hàm bạn được thực hiện như hàm thông thường.
- Trong *thân hàm bạn của một lớp có thể truy nhập tới các thuộc tính của đối tượng thuộc lớp này*. Đây là sự khác nhau duy nhất giữa hàm bạn và hàm thông thường.
- Hàm bạn có thể sử dụng chung cho nhiều lớp.

LỚP

HÀM BẠN

○ Khai báo hàm bạn:

- Cách 1: Dùng từ khóa *friend* để khai báo hàm trong lớp và xây dựng hàm bên ngoài như các hàm thông thường (không dùng từ khóa friend).
- Cách 2: Dùng từ khóa friend để xây dựng hàm trong định nghĩa lớp.

LỚP

HÀM BẠN

- Khai báo hàm bạn (cách 1):

```
class A
{
    private:
    // Khai báo các thuộc tính
    public:
    ...
    // Khai báo các hàm bạn của lớp A
    friend void f1 (...);
    friend double f2 (...);
    ...
};
```

LỚP

HÀM BẠN

- Khai báo hàm bạn (cách 1):

// Xây dựng các hàm f1,f2

```
void f1 (...)
```

```
{
```

```
    ...
```

```
}
```

```
double f2 (...)
```

```
{
```

```
    ...
```

```
}
```

LỚP

HÀM BẠN

○ Khai báo hàm bạn (cách 2):

```
class A
{
    ...
    // Khai báo các hàm bạn của lớp A
    friend void f1 (...)
    {
        ...
    }
    friend double f2 (...)
    {
        ...
    }
};
```

LỚP

HÀM BẠN

○ Ví dụ:

```
#include <iostream.h>
#include <conio.h>
class sophuc
{
    float a,b;
public:
    sophuc() {}
    sophuc(float x, float y)
    {a=x; b=y;}
    friend sophuc tong(sophuc,sophuc);
    friend void  hienthi(sophuc);
};
```


LỚP

HÀM BẠN

○ Ví dụ:

```
sophuc tong(sophuc c1, sophuc c2)
{
    sophuc c3;
    c3.a=c1.a + c2.a ;
    c3.b=c1.b + c2.b ;
    return (c3);
}

void hienthi(sophuc c)
{
    cout<<c.a<<" + "<<c.b<<"i"<<endl;
}
```

LỚP

HÀM BẠN

- Ví dụ:

```
main()
{
    clrscr();
    sophuc d1 (2.1,3.4);
    sophuc d2 (1.2,2.3) ;
    sophuc d3 ;
    d3 = tong(d1,d2);
    cout<<"d1= ";hienthi(d1);
    cout<<"d2= ";hienthi(d2);
    cout<<"d3= ";hienthi(d3);
    getch();
}
```

LỚP

HÀM TẠO

○ Khái niệm:

- Hàm tạo là một hàm thành phần đặc biệt của lớp làm nhiệm vụ tạo lập một đối tượng mới.
- Chương trình dịch sẽ cấp phát bộ nhớ cho đối tượng, sau đó sẽ gọi đến hàm tạo. Hàm tạo sẽ khởi gán giá trị cho các thuộc tính của đối tượng và có thể thực hiện một số công việc khác nhằm chuẩn bị cho đối tượng mới.

LỚP

HÀM TẠO

- Quy tắc xây dựng hàm tạo:
 - Tên hàm tạo trùng với tên của lớp.
 - Hàm tạo không có kiểu trả về.
 - Hàm tạo phải được khai báo trong vùng public.
 - Hàm tạo có thể được xây dựng bên trong hoặc bên ngoài định nghĩa lớp.
 - Hàm tạo có thể có tham số hoặc không có tham số.
 - Trong một lớp có thể có nhiều hàm tạo (cùng tên nhưng khác các tham số).

LỚP

HÀM TẠO

- Ví dụ:

```
class DIEM
{
private:
    int x,y;
public:
    DIEM()                //Ham tao khong tham so
    {
        x = y = 0;
    }
    DIEM(int x1, int y1)  //Ham tao co tham so
    {
        x = x1; y=y1;
    }
    //Cac thanh phan khac
};
```

LỚP HÀM TẠO

- Chú ý 1: Nếu lớp không có hàm tạo, chương trình dịch sẽ cung cấp một hàm tạo mặc định không đối, hàm này thực chất không làm gì cả. Như vậy một đối tượng tạo ra chỉ được cấp phát bộ nhớ, còn các thuộc tính của nó chưa được xác định.

LỚP

HÀM TẠO

○ Ví dụ:

```
#include <conio.h>
#include <iostream.h>
class DIEM
{
    private:
        int x,y;
    public:
        void in()
        {
            cout <<"\n" << x <<" " << y;
        }
};
```

LỚP

HÀM TẠO

○ Ví dụ:

```
main()
{
    DIEM d;
    d.in();
    DIEM *p;
    p= new DIEM [10];
    clrscr();
    d.in();
    for (int i=0; i<10; ++i)
        (p+i)->in();
    getch();
}
```


LỚP HÀM TẠO

○ Chú ý 2:

- Khi một đối tượng được khai báo thì hàm tạo của lớp tương ứng sẽ tự động thực hiện và khởi gán giá trị cho các thuộc tính của đối tượng. Dựa vào các tham số trong khai báo mà chương trình dịch sẽ biết cần gọi đến hàm tạo nào.
- Khi khai báo mảng đối tượng không cho phép dùng các tham số để khởi gán cho các thuộc tính của các đối tượng mảng.
- Câu lệnh khai báo một biến đối tượng sẽ gọi tới hàm tạo một lần.
- Câu lệnh khai báo một mảng n đối tượng sẽ gọi tới hàm tạo mặc định n lần.
- Với các hàm có các tham số kiểu lớp, thì chúng chỉ xem là các tham số hình thức, vì vậy khai báo tham số trong dòng đầu của hàm sẽ không tạo ra đối tượng mới và do đó không gọi tới các hàm tạo.

LỚP

HÀM TẠO

- Ví dụ:

```
#include <conio.h>
#include <iostream.h>
#include <iomanip.h>
class DIEM
{
    private:
    int x,y;
    public:
    DIEM()
    { x = y = 0; }
    DIEM(int x1, int y1)
    { x = x1; y = y1; }
    friend void in(DIEM d)
    { cout <<"\n" << d.x <<" " << d.y; }
    void in()
    { cout <<"\n" << x <<" " << y ; }
};
```

LỚP

HÀM TẠO

○ Ví dụ:

```
main()
{
    DIEM d1;
    DIEM d2(2,3);
    DIEM *d; d = new DIEM (5,6);
    clrscr();
    in(d1);           // Goi ham ban in()
    d2.in();          // Goi ham thanh phan in()
    in(*d);           // Goi ham ban in()
    DIEM(2,2).in();   // Goi ham thanh phan in()
    DIEM t[3];         // 3 lan goi ham tao khong doi
    DIEM *q;          // Goi ham tao khong doi
```

LỚP

HÀM TẠO

○ Ví dụ:

```
int n;  
cout << "\n N = "; cin >> n;  
q = new DIEM [n+1];          //n+1 lan goi  
//ham tao khong doi  
for (int i=0;i<=n;++i)  
    q[i]=DIEM (3+i,4+i);      //n+1 lan goi  
//ham tao co doi  
for (i=0;i<=n;++i)  
    q[i].in(); // Goi ham thanh phan in()  
for (i=0;i<=n;++i)  
    DIEM(5+i,6+i).in();       //Goi ham  
//thanh phan in()  
getch();  
}
```

LỚP HÀM TẠO

○ Chú ý 3:

- Nếu trong lớp đã có ít nhất một hàm tạo, thì hàm tạo mặc định sẽ không được phát sinh nữa.
- Khi đó mọi câu lệnh xây dựng đối tượng mới đều sẽ gọi đến một hàm tạo của lớp. Nếu không tìm thấy hàm tạo cần gọi thì chương trình dịch sẽ báo lỗi.
- Điều này thường xảy ra khi chúng ta không xây dựng hàm tạo không đối, nhưng lại sử dụng các khai báo không tham số.

LỚP

HÀM TẠO

○ Ví dụ:

```
#include <conio.h>
#include <iostream.h>
class DIEM
{
    private:
        int x,y;
    public:
        DIEM(int x1, int y1)
        {
            x=x1; y=y1;
        }
        void in()
        {
            cout << "\n" << x << " " << y << endl;
        }
};
```

LỚP

HÀM TẠO

- Ví dụ:

```
main()
{
    DIEM d1(200,200); // Goi ham tao co doi
    DIEM d2;           // Loi, goi ham tao khong doi
    d2= DIEM (3,5); // Goi ham tao co doi
    d1.in();
    d2.in();
    getch();
}
```

LỚP

HÀM TẠO

- Trong ví dụ trên, câu lệnh `DIEM d2;` trong hàm `main()` sẽ bị chương trình dịch báo lỗi. Bởi vì lệnh này sẽ gọi tới hàm tạo không đối, mà hàm tạo này chưa được xây dựng. Có thể khắc phục điều này bằng cách chọn một trong hai giải pháp sau:
 - Xây dựng thêm hàm tạo không đối.
 - Gán giá trị mặc định cho tất cả các đối `x1`, `y1` của hàm tạo đã xây dựng ở trên.

LỚP

HÀM TẠO

○ Ví dụ:

```
#include <conio.h>
#include <iostream.h>
class DIEM
{
    private:
        int x,y;
    public:
        DIEM(int x1=0, int y1=0)
        { x = x1; y = y1; }
        void in()
        { cout << "\n" << x << " " << y << " " << m; }
};
```

LỚP

HÀM TẠO

○ Ví dụ:

```
main()
{
    DIEM d1(2,3);    // Goi ham tao, khong dung
                    // tham so mac dinh
    DIEM d2;          //Goi ham tao,
                    //dung tham so mac dinh
    d2 = DIEM(6,7); // Goi ham tao,
                    // khong dung tham so mac
                    dinh
    d1.in();
    d2.in();
    getch();
}
```

LỚP

HÀM TẠO

○ Ví dụ:

```
main()
{
    DIEM d1(2,3);    // Goi ham tao, khong dung
                    // tham so mac dinh
    DIEM d2;          //Goi ham tao,
                    //dung tham so mac dinh
    d2 = DIEM(6,7); // Goi ham tao,
                    // khong dung tham so mac
                    dinh
    d1.in();
    d2.in();
    getch();
}
```

LỚP

HÀM HỦY

- Hàm huỷ là một hàm thành phần của lớp, có chức năng ngược với hàm tạo. Hàm huỷ được gọi trước khi giải phóng một đối tượng để thực hiện một số công việc có tính “dọn dẹp” trước khi đối tượng được huỷ bỏ, ví dụ giải phóng một vùng nhớ mà đối tượng đang quản lý, xoá đối tượng khỏi màn hình nếu như nó đang hiển thị...Việc huỷ bỏ đối tượng thường xảy ra trong 2 trường hợp sau:
- Trong các toán tử và hàm giải phóng bộ nhớ như delete, free...
- Giải phóng các biến, mảng cục bộ khi thoát khỏi hàm, phương thức.

LỚP

HÀM HỦY

- Hàm hủy mặc định:
 - Nếu trong lớp không định nghĩa hàm huỷ thì một hàm huỷ mặc định không làm gì cả được phát sinh.
 - Đối với nhiều lớp thì hàm huỷ mặc định là đủ, không cần đưa vào một hàm huỷ mới.

LỚP

HÀM HỦY

- Quy tắc viết hàm hủy:
 - Mỗi lớp chỉ có một hàm hủy.
 - Hàm hủy không có kiểu, không có giá trị trả về và không có tham số.
 - Tên hàm hủy có một dấu ngã ngay trước tên lớp.

LỚP

HÀM HỦY

○ Ví dụ:

```
class DATHUC
{
    private:
        int n;
        double *a;
    public:
        DATHUC()
        {
            n = 0; a = NULL;
        }
}
```

LỚP

HÀM HỦY

- Ví dụ:

```
class DATHUC
{
    private:
    int n; double *a;
    public:
    DATHUC(int n1)
    {
        n = n1; a = new double [n + 1];
    }
}
```


LỚP

HÀM HỦY

- Ví dụ:

```
class DATHUC
{
    ~DATHUC()
    {
        n = 0;
        delete a;
    }
};
```

LỚP

HÀM HỦY

- Ví dụ:

```
#include <iostream.h>
class Count
{
    private:
        static int counter;
        int obj_id;
    public:
        Count();
        static void display_total();
        void display();
        ~Count();
};
```

LỚP

HÀM HỦY

- Ví dụ:

```
int Count::counter;
Count::Count()
{
    counter++;
    obj_id = counter;
}
Count::~~Count()
{
    counter--;
    cout<<"Doi tuong "<<obj_id<<" duoc huy bo\n";
}
```

LỚP

HÀM HỦY

- Ví dụ:

```
void Count::display_total()
{
    cout << "Số các đối tượng được tạo ra là " << counter <<
        endl;
}

void Count::display()
{
    cout << "Object ID là " << obj_id << endl;
}
```

LỚP

HÀM HỦY

○ Ví dụ:

```
void main()
{
    Count a1;
    Count::display_total();
    Count a2, a3;
    Count::display_total();
    a1.display();
    a2.display();
    a3.display();
}
```

ĐỐI TƯỢNG BIẾN

- Đối tượng là biến thuộc kiểu lớp.
- Cú pháp khai báo biến đối tượng:
Tên_lớp Danh_sách_biến ;
- Đối tượng cũng có thể khai báo khi định nghĩa lớp theo cú pháp:

```
class tên_lớp  
{  
    ...  
} <Danh_sách_biến>;
```

ĐỐI TƯỢNG BIẾN

- Mỗi đối tượng sau khi khai báo sẽ được cấp phát một vùng nhớ riêng để chứa các thuộc tính của chúng.
- Không có vùng nhớ riêng để chứa các hàm thành phần cho mỗi đối tượng.
- Các hàm thành phần sẽ được sử dụng chung cho tất cả các đối tượng cùng lớp.

ĐỐI TƯỢNG BIẾN

- Truy nhập tới các thành phần của đối tượng:
 - Truy nhập đến dữ liệu thành phần:
Tên_đối_tượng.Tên_thuộc_tính
 - Chú ý:
 - Dữ liệu thành phần riêng chỉ có thể được truy nhập bởi những hàm thành phần của cùng một lớp
 - Đối tượng của lớp cũng không thể truy nhập.
 - Truy nhập đến hàm thành phần của lớp:
Tên_đối_tượng.Tên_hàm (Tham_số_thực_sự);

ĐỐI TƯỢNG BIẾN

- Ví dụ:

```
#include <conio.h>
#include <iostream.h>
class DIEM
{
    private:
        int x,y ;
    public:
        void nhapsl( )
        {
            cout << "\n Nhap hoành do va tung do cua diem:";
            cin >>x>>y ;
        }
        void hienthi( )
        {
            cout<<"\n x = " <<x<<" y = "<<y<<endl;
        }
};
```

ĐỐI TƯỢNG BIẾN

- Ví dụ:

```
//#include <graphics.h>
main()
{
    DIEM d1;
    d1.nhapsl();
    d1.hienthi ();
    getch();
    DIEM d2;
    d2.x = 10;
    d2.y = 20;
    d2.hienthi();
    getch();
}
```

ĐỐI TƯỢNG BIẾN

- Ví dụ: Viết chương trình nhập hai số nguyên, tìm giá trị lớn nhất của hai số nguyên đó.

```
#include <iostream>
#include <cstdlib>
#include <conio.h>
using namespace std;
class SN
{
    int m,n;
public :
    void nhap( )
    {
        cout<<endl<<"\nNhap hai so nguyen: " ;
        cin>>m>>n ;
    }
}
```

ĐỐI TƯỢNG BIẾN

- Ví dụ: Viết chương trình nhập hai số nguyên, tìm giá trị lớn nhất của hai số nguyên đó.

```
int max()
{
    return m>n?m:n;
}
void hienthi( )
{
    cout<<endl<<"Thanh phan du lieu lon nhat: "
    <<max()<<endl;}
int getm() {return m;}
int getn() {return n;}
};
```

ĐỐI TƯỢNG BIẾN

- Ví dụ: Viết chương trình nhập hai số nguyên, tìm giá trị lớn nhất của hai số nguyên đó.

```
main ()
{
    SN ob;
    ob.nhap();
    cout<<endl<<"So nguyen vua nhap la:"
    cout<<ob.getm()<<" - "<<ob.getn();
    ob.hienthi();
    getch();
}
```

ĐỐI TƯỢNG BIẾN

- Ví dụ: Viết chương trình xây dựng lớp số phức.
- `#include <iostream.h>`
- `#include <conio.h>`

```
class sophuc
{
    float a,b;
public:
    sophuc(){}
    sophuc(float x, float y)
    {a=x; b=y;}
    sophuc tong(sophuc,sophuc);
    void hienthi();
};
```

ĐỐI TƯỢNG BIẾN

- Ví dụ: Viết chương trình xây dựng lớp số phức.

```
sophuc sophuc::tong(sophuc c1, sophuc c2)
{
    sophuc c3;
    c3.a=c1.a + c2.a ;
    c3.b=c1.b + c2.b ;
    return (c3);
}

void sophuc::hienthi()
{
    cout<<a<<" + "<<b<<"i"<<endl;
}
```

ĐỐI TƯỢNG BIẾN

- Ví dụ: Viết chương trình xây dựng lớp số phức.

```
void main()
{
    clrscr();
    sophuc d1 (2.1,3.4);
    sophuc d2 (1.2,2.3) ;
    sophuc d3;
    d3 = d3.tong(d1,d2);
    cout<<"d1= ";d1.hienthi();
    cout<<"d2= ";d2.hienthi();
    cout<<"d3= ";d3.hienthi();
    getch();
}
```


ĐỐI TƯỢNG BIẾN

- Ví dụ: Viết chương trình xây dựng lớp phân số.

ĐỐI TƯỢNG

MẢNG VÀ CON TRỎ

✓ Con trỏ đối tượng:

- Mảng đối tượng chứa danh sách đối tượng.

Tên_lớp Tên_Mảng[n];

- Con trỏ đối tượng dùng để chứa địa chỉ của biến đối tượng, mảng đối tượng.
- Con trỏ được khai báo như sau:

Tên_lớp * Tên_con_trỏ;

ĐỐI TƯỢNG MẢNG VÀ CON TRỎ

✓ Ví dụ:

- Dùng lớp DIEM, ta có thể khai báo:
 - DIEM *p1, *p2, *p3 ; // Khai báo 3 con trỏ p1, p2, p3
 - DIEM d1, d2 ; // Khai báo hai đối tượng d1, d2
 - DIEM d [20] ; // Khai báo mảng đối tượng
- Có thể thực hiện câu lệnh :
 - p1 = &d2 ; // p1 chứa địa chỉ của d2, p1 trỏ tới d2
 - p2 = d ; // p2 trỏ tới đầu mảng d
 - p3 = new DIEM // tạo một đối tượng và chứa địa chỉ của nó vào p3

ĐỐI TƯỢNG

MẢNG VÀ CON TRỎ

- Để truy xuất các thành phần của lớp từ con trỏ đối tượng, ta viết như sau :
 - Tên_con_trỏ -> Tên_thuộc_tính
 - Tên_con_trỏ -> Tên_hàm(các tham số thực sự)
- Nếu con trỏ chứa đầu địa chỉ của mảng, có thể dùng con trỏ như tên mảng.

ĐỐI TƯỢNG MẢNG VÀ CON TRỎ

- Quy tắc sử dụng thuộc tính của mảng và con trỏ đối tượng:
 - Để truy xuất một thuộc tính của đối tượng sử dụng toán tử . hoặc toán tử ->.
 - Trong chương trình, không cho phép viết tên thuộc tính một cách đơn độc mà phải viết đi kèm tên đối tượng hoặc tên con trỏ theo các mẫu sau:
 - `ten_doi_tuong.ten_thuoc_tinh`
 - `ten_con_trỏ->ten_thuoc_tinh`
 - `ten_mang_doi_tuong[chi_so].ten_thuoc_tinh`
 - `ten_con_trỏ[chi_so].ten_thuoc_tinh`

ĐỐI TƯỢNG

MẢNG VÀ CON TRỎ

- Ví dụ: Viết chương trình quản lý mặt hàng gồm mã hàng và đơn giá bằng định nghĩa lớp.

```
#include <iostream.h>
```

```
#include <conio.h>
```

```
class mhang
```

```
{
```

```
    int maso;
```

```
    float gia;
```

ĐỐI TƯỢNG

MẢNG VÀ CON TRỎ

- Ví dụ: Viết chương trình quản lý mặt hàng gồm mã hàng và đơn giá bằng định nghĩa lớp.

```
public:
    void Setdata(int a, float b)
    {
        maso = a; gia = b;
    }
    void show()
    {
        cout << "\nMã số: " << maso << endl;
        cout << "Giá: " << gia << endl;
    }
};
```

ĐỐI TƯỢNG

MẢNG VÀ CON TRỎ

- Ví dụ: Viết chương trình quản lý mặt hàng gồm mã hàng và đơn giá bằng định nghĩa lớp.

```
main()
{   clrscr();
    int k;
    cout<<"\nNhap tong so hang mat hang:"; cin>>k;
    mhang *p = new mhang[k];
    mhang *d = p;
    int x,i;
    float y;
    cout<<"\nNhap vao du lieu "<<k<<" mat hang :";
```


ĐỐI TƯỢNG

MẢNG VÀ CON TRỎ

- Ví dụ: Viết chương trình quản lý mặt hàng gồm mã hàng và đơn giá bằng định nghĩa lớp.

```
for (i = 0; i < k; i++)  
{  
    cout << "\nNhap ma hang va don gia cho  
    mat hang thu " << i + 1;  
    cin >> x >> y;  
    p -> Setdata(x, y);  
    p++;  
}
```

ĐỐI TƯỢNG

MẢNG VÀ CON TRỎ

- Ví dụ: Viết chương trình quản lý mặt hàng gồm mã hàng và đơn giá bằng định nghĩa lớp.

```
for (i = 0; i < k; i++)  
{  
    cout<<"\nMat hang thu: "<<(i+1)<<endl;  
    d -> show();  
    d++;  
}  
getch();  
}
```

BÀI TẬP CHƯƠNG 2

- **Bài tập 1:** Xây dựng lớp thời gian **Time**. Dữ liệu thành phần bao gồm giờ, phút giây. Các hàm thành phần bao gồm: hàm tạo, hàm truy cập dữ liệu, hàm normalize() để chuẩn hóa dữ liệu nằm trong khoảng quy định của giờ ($0 \leq \text{giờ} < 24$), phút ($0 \leq \text{phút} < 60$), giây ($0 \leq \text{giây} < 60$), hàm advance(int h, int m, int s) để tăng thời gian hiện hành của đối tượng đang tồn tại, hàm reset(int h, int m, int s) để chỉnh lại thời gian hiện hành của một đối tượng đang tồn tại và một hàm print() để hiển thị dữ liệu.

BÀI TẬP CHƯƠNG 2

- **Bài tập 2:** Xây dựng lớp **Date**. Dữ liệu thành phần bao gồm ngày, tháng, năm. Các hàm thành phần bao gồm: hàm tạo, hàm nhập dữ liệu, hàm `normalize()` để chuẩn hóa dữ liệu nằm trong khoảng quy định của ngày ($1 \leq \text{ngày} < \text{daysIn}(\text{tháng})$), tháng ($1 \leq \text{tháng} < 12$), năm ($\text{năm} \geq 1$), hàm `daysIn(int)` trả về số ngày trong tháng, hàm `advance(int y, int m, int d)` để tăng ngày hiện lên các năm y, tháng m, ngày d của đối tượng đang tồn tại, hàm `reset(int y, int m, int d)` để đặt lại ngày cho một đối tượng đang tồn tại và một hàm `print()` để hiển thị dữ liệu.

BÀI TẬP CHƯƠNG 2

- **Bài tập 3:** Thực hiện một lớp **String**. Mỗi đối tượng của lớp sẽ đại diện một chuỗi ký tự. Những thành phần dữ liệu là chiều dài chuỗi, và chuỗi ký tự. Các hàm thành phần bao gồm: hàm tạo, hàm nhập chuỗi, hàm hiển thị chuỗi, hàm `character(int i)` trả về một ký tự trong chuỗi được chỉ định bằng tham số `i`.

ĐỐI TƯỢNG

TOÁN TỬ TẢI BỘI (OPERATOR OVERLOADING)

- Định nghĩa: Các toán tử cùng tên thực hiện nhiều chức năng khác nhau được gọi là toán tử tải bội.
- Dạng định nghĩa tổng quát của toán tử tải bội như sau:

```
Kiểu_trả_về      operator op(danh sách tham số)
{
    //thân toán tử
}
```

- Trong đó:
 - Kiểu_trả_về là kiểu kết quả thực hiện của toán tử.
 - op là tên toán tử tải bội.

ĐỐI TƯỢNG

TOÁN TỬ TẢI BỘI (OPERATOR OVERLOADING)

○ Định nghĩa:

operator op (danh sách tham số) gọi là hàm toán tử tải bội, nó có thể là hàm thành phần hoặc là hàm bạn, nhưng không thể là hàm tĩnh.

ĐỐI TƯỢNG

TOÁN TỬ TẢI BỘI (OPERATOR OVERLOADING)

- Danh sách tham số được khai báo tương tự khai báo biến nhưng phải tuân theo những quy định sau:
 - Nếu toán tử tải bội là hàm thành phần thì: không có tham số cho toán tử một ngôi và một tham số cho toán tử hai ngôi. Cũng giống như hàm thành phần thông thường, hàm thành phần toán tử có đối đầu tiên (không tường minh) là con trỏ this.
 - Nếu toán tử tải bội là hàm bạn thì: có một tham số cho toán tử một ngôi và hai tham số cho toán tử hai ngôi.

ĐỐI TƯỢNG

TOÁN TỬ TẢI BỘI (OPERATOR OVERLOADING)

- **Quá trình xây dựng toán tử tải bội được thực hiện như sau:**

- Định nghĩa lớp để xác định kiểu dữ liệu sẽ được sử dụng trong các toán tử tải bội
- Khai báo hàm toán tử tải bội trong vùng public của lớp
- Định nghĩa nội dung cần thực hiện

ĐỐI TƯỢNG

TOÁN TỬ TẢI BỘI (OPERATOR OVERLOADING)

- Chú ý:
 - Trong C++ ta có thể đưa ra nhiều định nghĩa mới cho hầu hết các toán tử trong C++, ngoại trừ những toán tử sau đây:
 - Toán tử xác định thành phần của lớp (‘.’)
 - Toán tử phân giải miền xác định (‘::’)
 - Toán tử xác định kích thước (‘sizeof’)
 - Toán tử điều kiện 3 ngôi (‘?:’)

ĐỐI TƯỢNG

TOÁN TỬ TẢI BỘI (OPERATOR OVERLOADING)

○ Chú ý:

- Mặc dầu ngữ nghĩa của toán tử được mở rộng nhưng cú pháp, các quy tắc văn phạm như số toán hạng, quyền ưu tiên và thứ tự kết hợp thực hiện của các toán tử vẫn không có gì thay đổi.
- Không thể thay đổi ý nghĩa cơ bản của các toán tử đã định nghĩa trước, ví dụ không thể định nghĩa lại các phép toán $+$, $-$ đối với các số kiểu `int`, `float`.
- Các toán tử `=`, `()`, `[]`, `->` yêu cầu hàm toán tử phải là hàm thành phần của lớp, không thể dùng hàm bạn để định nghĩa toán tử tải bội.

ĐỐI TƯỢNG

TOÁN TỬ TẢI BỘI (OPERATOR OVERLOADING)

- Ví dụ: Xây dựng lớp số phức

```
#include <iostream.h>
```

```
#include <conio.h>
```

```
class sophuc
```

```
{
```

```
    float a,b;
```

```
    public : sophuc() {}
```

```
    sophuc(float x, float y)
```

```
    {a=x; b=y;}
```

ĐỐI TƯỢNG

TOÁN TỬ TẢI BỘI (OPERATOR OVERLOADING)

○ Ví dụ: Xây dựng lớp số phức

```
friend sophuc operator +(sophuc c1, sophuc c2)
{
    sophuc c3;
    c3.a= c1.a + c2.a ;
    c3.b= c1.b + c2.b ;
    return (c3);
}
void hienthi(sophuc c)
{
    cout<<c.a<<" + "<<c.b<<"i"<<endl;
}
};
```

ĐỐI TƯỢNG

TOÁN TỬ TẢI BỘI (OPERATOR OVERLOADING)

- Ví dụ: Xây dựng lớp số phức

```
main()
```

```
{
```

```
    clrscr();
```

```
    sophuc d1 (2.1,3.4);
```

```
    sophuc d2 (1.2,2.3) ;
```

```
    sophuc d3 ;
```

```
    d3 = d1+d2; //d3=d1.operator +(d2);
```

```
    cout<<"d1= ";d1.hienthi(d1);
```

```
    cout<<"d2= ";d2.hienthi(d2);
```

```
    cout<<"d3= ";d3.hienthi(d3);
```

```
    getch();
```

```
}
```

ĐỐI TƯỢNG

TOÁN TỬ TẢI BỘI (OPERATOR OVERLOADING)

- Ví dụ: Toán tử tải bội trên lớp chuỗi ký tự

```
#include <iostream.h>
#include <string.h>
#include <conio.h>
class string
{
    char s[80];
public:
    string() { *s='\0'; }
    string(char *p) { strcpy(s,p); }
    char *get() { return s;}
    string operator + (string s2);
    string operator = (string s2);
    int operator < (string s2);
    int operator > (string s2);
    int operator == (string s2);
};
```

ĐỐI TƯỢNG

TOÁN TỬ TẢI BỘI (OPERATOR OVERLOADING)

- Ví dụ: Toán tử tải bội trên lớp chuỗi ký tự

```
string string::operator +(string s2)
{
    strcat(s,s2.s);
    return *this ;
}
string string::operator =(string s2)
{
    strcpy(s,s2.s);
    return *this;
}
```


ĐỐI TƯỢNG

TOÁN TỬ TẢI BỘI (OPERATOR OVERLOADING)

- Ví dụ: Toán tử tải bội trên lớp chuỗi ký tự

```
int string::operator <(string s2)
{
    return strcmp(s,s2.s)<0 ;
}
int string::operator >(string s2)
{
    return strcmp(s,s2.s)>0 ;
}
```

ĐỐI TƯỢNG

TOÁN TỬ TẢI BỘI (OPERATOR OVERLOADING)

- Ví dụ: Toán tử tải bội trên lớp chuỗi ký tự

```
int string::operator ==(string s2)
{
    return strcmp(s,s2.s)==0 ;
}
```

ĐỐI TƯỢNG

TOÁN TỬ TẢI BỘI (OPERATOR OVERLOADING)

- Ví dụ: Toán tử tải bội trên lớp chuỗi ký tự

```
main()
```

```
{
```

```
    clrscr();
```

```
    string o1 ("Trung Tam "), o2 (" Tin hoc"), o3;
```

```
    cout<<"o1 = "<<o1.get()<<'\n';
```

```
    cout<<"o2 = "<<o2.get()<<'\n';
```

```
    if (o1 > o2)
```

```
        cout << "o1 > o2 \n";
```

```
    if (o1 < o2)
```

```
        cout << "o1 < o2 \n";
```

```
    if (o1 == o2)
```

```
        cout << "o1 bang o3 \n";
```

ĐỐI TƯỢNG

TOÁN TỬ TẢI BỘI (OPERATOR OVERLOADING)

- Ví dụ: Toán tử tải bội trên lớp chuỗi ký tự

```
o3=o1+o2;
cout<<"o3 ="<<o3.get()<<'\\n';    //Trung tam
    tin hoc
o3=o2;
cout<<"o2 = "<<o2.get()<<'\\n'; //Tin hoc
cout<<"o3 = "<<o3.get()<<'\\n'; //Tin hoc
if (o2 == o3)
    cout << "o2 bang o3 \\n";
getch();
}
```

THÀNH PHẦN TĨNH

DỮ LIỆU

- Dữ liệu thành phần tĩnh được cấp phát một vùng nhớ cố định.
- Dữ liệu thành phần tĩnh tồn tại ngay cả khi lớp chưa có một đối tượng nào cả.
- Dữ liệu thành phần tĩnh là chung cho cả lớp, nó không phải là riêng của mỗi đối tượng.

THÀNH PHẦN TĨNH

DỮ LIỆU

- Dữ liệu thành phần tĩnh được khai báo bằng từ khoá ***static***.

```
class A
{
    private:
        static int ts; // Thành phần tĩnh
        int x;
    ...
};
```

THÀNH PHẦN TĨNH DỮ LIỆU

- Khi đó, khai báo:
A u, v; // Khai báo 2 đối tượng
- Có nghĩa là:
 - u.x và v.x có 2 vùng nhớ khác nhau.
 - u.ts và v.ts chỉ là một, chúng cùng biểu thị một vùng nhớ
 - Thành phần ts tồn tại ngay khi u và v chưa khai báo.
 - Biểu thị thành phần tĩnh: A::ts

THÀNH PHẦN TĨNH

KHAI BÁO VÀ KHỞI GÁN GIÁ TRỊ CHO THÀNH PHẦN TĨNH

- Thành phần tĩnh sẽ được cấp phát bộ nhớ và khởi gán giá trị đầu bằng một câu lệnh khai báo đặt sau định nghĩa lớp.
- Cú pháp:
 - `int A::ts;       //Khởi gán cho ts giá trị 0`
 - `int A::ts=12; //Khởi gán cho ts giá trị 12`
- Chú ý: Khi chưa khai báo thì thành phần tĩnh chưa tồn tại.

THÀNH PHẦN TĨNH

PHƯƠNG THỨC (HÀM)

- Hàm thành phần tĩnh là chung cho toàn bộ lớp và không lệ thuộc vào một đối tượng cụ thể.
- Hàm thành phần tồn tại ngay khi lớp chưa có đối tượng nào.
- Trong thân hàm thành phần tĩnh chỉ cho phép truy nhập đến dữ liệu thành phần tĩnh.

THÀNH PHẦN TĨNH

VÍ DỤ

- Ví dụ:

```
#include <conio.h>
#include <iostream.h>
class HDBH
{
    private:
    int shd;
    char *tenhang;
    double tienban;
    static int tshd;
    static double tstienban;
```

THÀNH PHẦN TĨNH

VÍ DỤ

- Ví dụ (tiếp):

```
public:
    static void in()
    {
        cout <<"\n" <<tshd;
        cout <<"\n" <<tstienban;
    }
};
//Khai báo và khởi gán biến tĩnh tshd
int HDBH::tshd=5;
//Khai báo và khởi gán biến tĩnh tstienban
double HDBH::tstienban=20000.0;
```

THÀNH PHẦN TĨNH

VÍ DỤ

- Ví dụ (tiếp):

```
void main()  
{  
    HDBH::in();  
    getch();  
}
```

BÀI TẬP CHƯƠNG 2

- **Bài tập 4:** Xây dựng lớp **Sinhvien** để quản lý họ tên sinh viên, năm sinh, điểm thi 9 môn học của các sinh viên. Cho biết sinh viên nào được làm khóa luận tốt nghiệp, bao nhiêu sinh viên thi tốt nghiệp, bao nhiêu sinh viên thi lại, tên môn thi lại. Tiêu chuẩn để xét như sau:
 - Sinh viên làm khóa luận phải có điểm trung bình từ 7 trở lên, trong đó không có môn nào dưới 5.
 - Sinh viên thi tốt nghiệp khi điểm trung bình nhỏ hơn 7 và điểm các môn không dưới 5.
 - Sinh viên thi lại môn dưới 5.

BÀI TẬP CHƯƠNG 2

- **Bài tập 5:** Viết chương trình thực hiện các yêu cầu sau đây:
 - Nhập dữ liệu cho các hóa đơn (dùng cấu trúc danh sách liên kết đơn), các thông tin của hóa đơn bao gồm: số hóa đơn, tên hàng, tiền bán.
 - Chương trình có sử dụng toán tử new và delete.
 - In ra danh sách hóa đơn có sắp xếp theo thứ tự giảm dần của tiền bán.
 - Cho biết tổng số hóa đơn và tổng số tiền bán.

MỤC TIÊU

- Trình bày phương pháp định nghĩa lớp, thành phần lớp, phạm vi truy xuất thành phần, các hàm đặc biệt của lớp: hàm bạn, hàm tạo và hàm hủy.
- Khai báo đối tượng, mảng và con trỏ đối tượng.
- Cách thức truy nhập đến các thành phần của đối tượng, con trỏ mà mảng đối tượng.
- Trình bày khái niệm thành phần tĩnh, dữ liệu và phương thức tĩnh.

TÓM TẮT CHƯƠNG 2

- Định nghĩa lớp, khai báo các thành phần dữ liệu, định nghĩa thành phần phương thức, từ khóa xác định phạm vi truy xuất, con trỏ this. Định nghĩa hàm bạn, hàm tạo và hàm hủy.
- Khai báo biên đối tượng, mảng và con trỏ đối tượng. Truy nhập đến các thành phần của đối tượng. Toán tử tải bội (Operator overloading).
- Khái niệm thành phần tĩnh, dữ liệu và phương thức tĩnh.